

Instalación de ROS 2

Para utilizar ROS 2 se plantean los siguientes casos de uso:

- Nativo en Linux
- Windows + WSL
- Windows + Máquina Virtual con Linux

La mejor opción es utilizar Linux de forma nativa, ya que hay mayor compatibilidad con el software de ROS y se le puede sacar máximo provecho.

Windows es soportado oficialmente por ROS 2, y funciona plenamente en términos de comunicación y arquitectura. Sin embargo, el soporte de paquetes y drivers es más limitado respecto a Linux; Gazebo y otros simuladores tienen soporte parcial o indirecto; y Windows en sí mismo no es una plataforma de tiempo real.

Es por ello que en el presente curso utilizaremos **Ubuntu 24.04 LTS (Noble)** con **ROS 2 Jazzy**, ya sea en su forma nativa (recomendado) como a través de WSL o máquinas virtuales.

El uso de máquinas virtuales o WSL es útil para el desarrollo rápido, ya que evita tener que cambiar de OS si Windows es el utilizado habitualmente. Sin embargo, a la hora de realizar simulaciones o comunicarse con otros dispositivos estas alternativas pueden traer más dificultades que el uso de Linux de forma nativa.

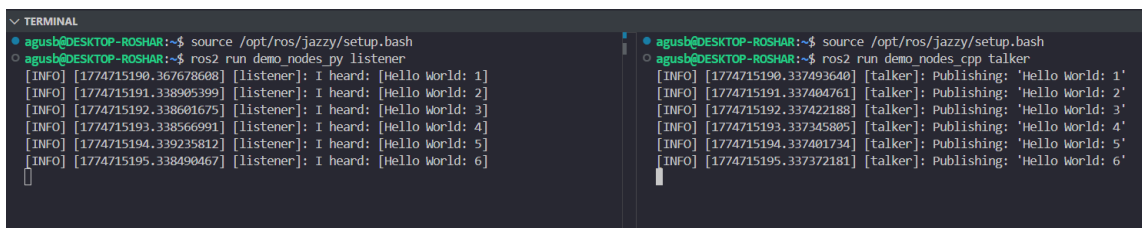
Instalación

Para la instalación recomendamos seguir el tutorial oficial de ROS en [esta página](#) (si se quiere instalar ROS en un entorno de WSL, primero se debe instalar WSL siguiendo el tutorial de la próxima sección).

La instalación de ROS 2 básicamente consta de los siguientes puntos:

1. Asegurarse que tu sistema esté utilizando un locale soportado por UTF-8.
2. Habilitar y configurar los repositorios apt de ROS 2 en el sistema.
3. Actualizar el sistema con apt update y apt upgrade, luego realizar **instalación “Desktop”** de ROS2 (ROS, RViz, demos, tutoriales).
4. Instalar las herramientas de desarrollador: **ros-dev-tools** (Compilers and other tools to build ROS packages)

5. Validar que todo esté correctamente instalado haciendo source del entorno de ROS y luego ejecutando los nodos de ejemplo (*talker* y *listener*).



```

TERMINAL
agusb@DESKTOP-ROSHAR:~$ source /opt/ros/jazzy/setup.bash
agusb@DESKTOP-ROSHAR:~$ ros2 run demo_nodes_py listener
[INFO] [1774715190.367678608] [listener]: I heard: [Hello World: 1]
[INFO] [1774715191.338905399] [listener]: I heard: [Hello World: 2]
[INFO] [1774715192.338601675] [listener]: I heard: [Hello World: 3]
[INFO] [1774715193.338566991] [listener]: I heard: [Hello World: 4]
[INFO] [1774715194.339235812] [listener]: I heard: [Hello World: 5]
[INFO] [1774715195.338490467] [listener]: I heard: [Hello World: 6]

agusb@DESKTOP-ROSHAR:~$ source /opt/ros/jazzy/setup.bash
agusb@DESKTOP-ROSHAR:~$ ros2 run demo_nodes_cpp talker
[INFO] [1774715190.337493640] [talker]: Publishing: 'Hello World: 1'
[INFO] [1774715191.337404761] [talker]: Publishing: 'Hello World: 2'
[INFO] [1774715192.337422188] [talker]: Publishing: 'Hello World: 3'
[INFO] [1774715193.337345805] [talker]: Publishing: 'Hello World: 4'
[INFO] [1774715194.337401734] [talker]: Publishing: 'Hello World: 5'
[INFO] [1774715195.337372181] [talker]: Publishing: 'Hello World: 6'

```

6. [Opcional] Modificar `/.bashrc` para sourcear el underlay de ROS siempre que se abra la terminal.

Instalar cualquier paquete de ROS 2 a través de APT

Siempre que se necesite instalar un paquete particular de ROS2, es posible hacerlo con el siguiente comando:

```
$ sudo apt update
$ sudo apt install ros-<ROS-DISTRO>-<NOMBRE_PAQUETE>
```

rosdep

Se recomienda además instalar la herramienta **rosdep**, una utilidad de gestión de dependencias para paquetes y librerías externas. Es capaz de identificar e instalar dependencias faltantes, generalmente usado antes de construir un workspace.

Como **rosdep** ya viene instalado con ROS 2, sólo hay que inicializarlo con los siguientes comandos:

```
$ sudo rosdep init
$ rosdep update
```

Luego, dentro del workspace donde se quieren instalar los paquetes:

```
$ rosdep install --from-paths src --ignore-src -y -r
```

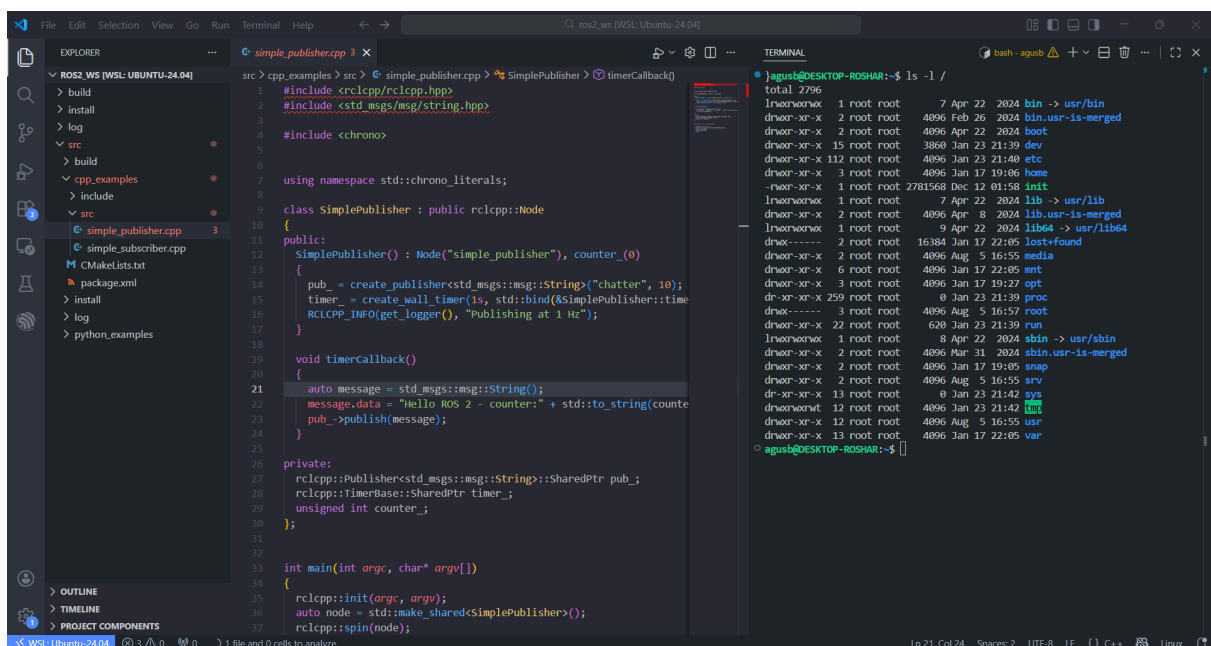
Instalación de WSL

Es posible instalar WSL fácilmente siguiendo el [tutorial oficial de Microsoft](#). En resumen, éste consiste de los siguientes pasos:

1. Instalar WSL usando PowerShell
2. Instalar la distro de Linux deseada (Ubuntu-24.04)
3. Generar usuario y contraseña del usuario
4. [Opcional] Instalar la extensión de WSL en VSCode

Para poder conectar dispositivos USB a WSL (se necesita más adelante al trabajar con microROS) seguir [este tutorial](#).

De esta forma, se puede trabajar en Windows 10/11 con un entorno Linux corriendo encima. Integrándolo con VSCode es posible visualizar su sistema de archivos y correr programas en la terminal integrada:



The screenshot shows the Visual Studio Code interface with the WSL (Ubuntu-24.04) environment. The Explorer panel on the left shows the file structure of the project, including 'src' and 'include' folders. The main editor displays the code for 'simple_publisher.cpp', which includes ROS2 headers and implements a simple publisher class. The terminal window on the right shows the output of the 'ls -l' command, displaying system files and directories with their permissions, owners, sizes, and modification dates.

```
src > cpp_examples > src > simple_publisher.cpp > SimplePublisher > timerCallback
```

```
#include <rclcpp/rclcpp.hpp>
#include <std_msgs/msg/string.hpp>
#include <chrono>

using namespace std::chrono_literals;

class SimplePublisher : public rclcpp::Node
{
public:
    SimplePublisher() : Node("simple_publisher"), counter_(0)
    {
        pub_ = create_publisher<std_msgs::msg::String>("chatter", 10);
        timer_ = create_wall_timer(1s, std::bind(&SimplePublisher::timer_callback, this));
        RCLCPP_INFO(get_logger(), "Publishing at 1 Hz");
    }

    void timer_callback()
    {
        auto message = std_msgs::msg::String();
        message.data = "Hello ROS 2 - counter: " + std::to_string(counter_);
        pub_->publish(message);
        counter_++;
    }

private:
    rclcpp::Publisher<std_msgs::msg::String>::SharedPtr pub_;
    rclcpp::TimerBase::SharedPtr timer_;
    unsigned int counter_;
};

int main(int argc, char* argv[])
{
    rclcpp::init(argc, argv);
    auto node = std::make_shared<SimplePublisher>();
    rclcpp::spin(node);
}
```

```
agush@DESKTOP-ROSHAR:~$ ls -l /
total 2796
lrwxrwxrwx 1 root root      7 Apr 22 2024 bin -> usr/bin
drwxr-xr-x 2 root root    4096 Feb 26 2024 bin.usr-is-merged
drwxr-xr-x 2 root root    4096 Apr 22 2024 boot
drwxr-xr-x 15 root root   3808 Jan 23 21:39 dev
drwxr-xr-x 112 root root  4096 Jan 23 21:40 etc
drwxr-xr-x 3 root root    4096 Jan 17 19:06 home
-rwxr-xr-x 1 root root 2781568 Dec 12 01:58 init
lrwxrwxrwx 1 root root      7 Apr 22 2024 lib -> usr/lib
drwxr-xr-x 2 root root    4096 Apr  8 2024 lib.usr-is-merged
lrwxrwxrwx 1 root root      9 Apr 22 2024 lib64 -> usr/lib64
drwx----- 2 root root 16384 Jan 17 22:05 lost-found
drwxr-xr-x 2 root root    4096 Aug  5 16:55 media
drwxr-xr-x 6 root root    4096 Jan 17 22:05 mnt
drwxr-xr-x 3 root root    4096 Jan 17 19:27 opt
dr-xr-xr-x 259 root root      0 Jan 23 21:39 proc
drwx----- 3 root root    4096 Aug  5 16:57 root
drwxr-xr-x 22 root root    628 Jan 23 21:39 run
lrwxrwxrwx 1 root root      8 Apr 22 2024/sbin -> usr/sbin
drwxr-xr-x 2 root root    4096 Mar 31 2024/sbin.usr-is-merged
drwxr-xr-x 2 root root    4096 Jan 17 19:05 snap
drwxr-xr-x 2 root root    4096 Aug  5 16:55 srv
dr-xr-xr-x 13 root root      0 Jan 23 21:42 sys
drwxrwxrwt 12 root root    4096 Jan 23 21:42 tmp
drwxr-xr-x 12 root root    4096 Aug  5 16:55 usr
drwxr-xr-x 13 root root    4096 Jan 17 22:05 var
```